

TELECOMUNICAÇÕES

ENG. ELETROTÉCNICA E DE COMPUTADORES

Relatório da 2ª parte do trabalho laboratorial

Beatriz Coelho ist 1107113
Carolina Baltazar ist 1107117
Tânia Ranchordas ist 1107474

1º Semestre 2024/2025 – P1

Índice

Introdução	2
Desmodulador BPSK	2
Desmodulador QPSK	2
Análise das constelações	4
Script e análise de erros	5
Análise BPSK	7
Análise QPSK	8
Conclusão	9

Introdução

Neste trabalho de laboratório implementamos e estudamos os modeladores digitais BPSK e QPSK, analisando o seu funcionamento, bem como as vantagens e desvantagens de cada um. A análise concentrou-se na deteção de erros em relação às variações nos parâmetros **Noise Voltage** e **Loop Bandwidth**. Adicionalmente, foi montado e testado um desmodulador QPSK no sistema **GNURadio Companion**, correspondente ao modulador QPSK, que separa uma sequência de bytes em quatro grupos de dois bits que codificam um dos símbolos da constelação QPSK. Finalmente, foi investigado o valor ideal para o parâmetro **Loop Bandwidth** no módulo **Symbol Sync** para minimizar erros de deteção, considerando a variação do ruído, tanto para BPSK como para QPSK.

Desmodulador BPSK

O **BPSK** (*Binary Phase Shift Keying*) é uma técnica simples de modulação digital onde a onda portadora muda para representar bits de dados. Neste caso, a fase portadora altera-se em 180° para distinguir os valores "0" e "1": o bit "1" mantém a fase original, enquanto o bit "0" altera-a em 180° . Isto cria uma representação onde a fase está "desfasada", sendo uma forma simples de **PSK** (*Phase Shift Keying*), que modula apenas um bit por símbolo, representando assim um formato digital de modulação eficiente e simples.

A expressão matemática que representa o **BPSK** é:

$$s_i(t) = \begin{cases} \sqrt{\frac{2E_b}{T_b}} \cos(2\pi f_c t) & \text{para o símbolo 1} \\ -\sqrt{\frac{2E_b}{T_b}} \cos(2\pi f_c t) & \text{para o símbolo 0} \end{cases}$$

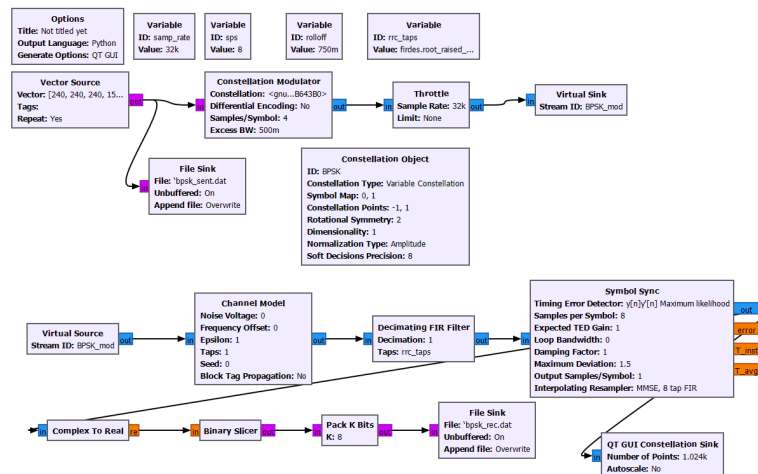


Figura 1: Flowgraph do desmodulador BPSK

Desmodulador QPSK

O **QPSK** (*Quadrature Phase Shift Keying*), apesar de ser uma modulação **PSK** semelhante a **BPSK**, traz uma complexidade adicional, pois analisa pares de bits simultaneamente ao

invês de bits individuais para determinar o *phase shift* do *carrier signal*. No **QPSK**, o sinal portador sofre um desvio de fase específico que depende do par de bits recebidos. Com isso, a transmissão de dados acontece em grupos de dois bits, o que permite aumentar a eficiência espectral.

A constelação **QPSK** é formada por quatro pontos que estão igualmente distribuídos ao longo de uma circunferência. Cada ponto da constelação representa um símbolo com uma fase específica e corresponde a um par de bits codificado em Gray, onde bits adjacente diferem em apenas um valor. As coordenadas da constelação para os símbolos usados são: $[-1-1j, -1+1j, +1+1j, +1-1j] \rightarrow [00, 01, 11, 10]$

Assim, cada símbolo do **QPSK** tem um valor complexo, representado pelas coordenadas da constelação. Isto permite que dois bits sejam transmitidos por símbolo, duplicando a eficiência de uso de banda em comparação com o **BPSK**. A expressão matemática que representa o **QPSK** é dada por:

$$s_i(t) = \sqrt{\frac{2E_b}{T_b}} \cos \left(2\pi f_c t + \frac{\pi}{4}(2i - 1) \right), \quad i = 1, 2, 3, 4$$

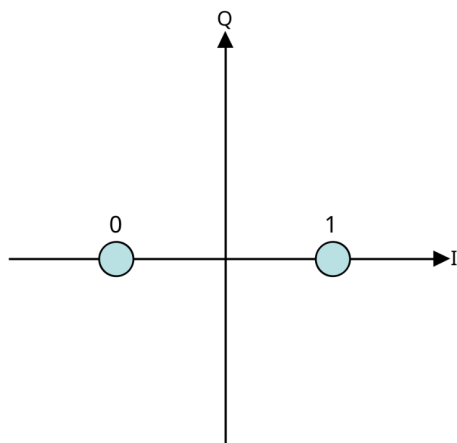


Figura 2: BPSK

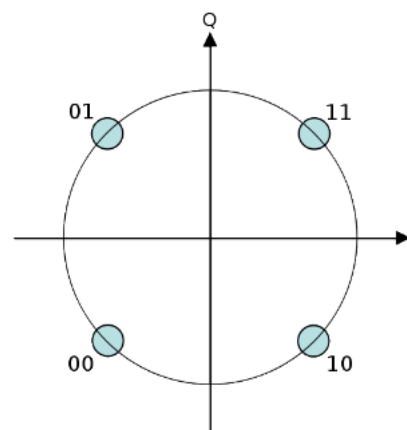


Figura 3: QPSK

A partir do flowgraph do desmodulador **BPSK** foi nos possível a montagem do desmodulador **QPSK**. Primeiramente, começamos por alterar o número de Samples no *Constellation Modulator* para o dobro. No bloco *Constellation Object* também fizemos alterações de forma estarem de acordo com o modelo teórico do **QPSK**. Assim, o vetor *Symbol Map* passou a ter quatro valores, temos agora quatro pontos, $[0, 1, 3, 2]$. Ao mesmo tempo, o parametro *Constellation Points*, para ficar de acordo com a figura x, passou a ser: $[-1-1j, -1+1j, 1-1j, 1+1j]$. Como temos agora o dobro dos pontos o parâmetro *Rotational Symmetry* passou para o dobro, 4 temos agora parte real e parte imaginária.

Após o bloco *Symbol Sync* precisamos de dividir o sinal complexo em duas partes: a parte real e a parte imaginária. Para isso utilizamos dois blocos distintos: o bloco *Complex to Real* extrai a parte real do sinal e o bloco *Complex to Imag* extrai a parte imaginária. Desta forma

conseguimos converter o vetor inicial $[-1, 1]$ para uma representação complexa $[-1-1j, -1+1j, 1-1j, 1+1j]$. Após esta separação, aplicamos o *Binary Slicer* a cada uma das saídas para converter o sinal positivo em '1' e o negativo em '0'.

De seguida usamos o bloco *Interleave* que tem como objetivo de intercalar os elementos de cada uma das entradas. Desta forma, a saída alterna entre bits da parte real e da parte imaginária. Por fim, utilizamos o bloco *Pack K Bits*, que agrupa os bits da saída, 4 da parte real e 4 da parte imaginária, para formar bytes de 8 bits. Estes blocos são direccionados ao arquivo de saída *qpsk.ec* através do bloco *File Sink*.

Tanto para este desmodulador como para o BPSK usamos no Vector Source um vetor personalizado com os nossos números de aluno: $[240, 240, 240, 15, 15, 15, 240, 240, 240] + [10, 71, 17, 10, 74, 74, 10, 71, 13] + [15, 15, 240, 240, 240, 15, 15, 15, 0, 0, 0, 0, 0, 0]$



Figura 4: QPSK

Análise das constelações

As figuras seguintes mostram o comportamento das constelações **BPSK** e **QPSK** em diferentes condições de *Noise Voltage* e *Loop Bandwidth*. Nas figuras 4 e 5 tanto o *Noise Voltage* como o *Loop Bandwidth* estão fixados a 0. Nestas condições, na constelação **BPSK** é fácil identificar os dois estados de fase, 0° e 180° , já que os pontos estão dispostos apenas ao longo do eixo real, sem componente imaginária. Na constelação **QPSK** observamos os quatro símbolos nas fases $\frac{\pi}{4}$, $\frac{3\pi}{4}$, $\frac{5\pi}{4}$, $\frac{7\pi}{4}$. Ao analisar as figuras podemos confirmar que o ruído é um dos principais fatores limitantes em sistemas de comunicação. Isto comprova-se nas figuras 6 e 7, onde as constelações são distorcidas devido ao aumento do valor de *Noise Voltage*. O aumento do nível de ruído tem como consequência o aumento dos erros na deteção. Finalmente, nas figuras 8 e 9, ao intensificarmos ainda mais o *Noise Voltage*, o ruído torna-se ainda mais evidente, confirmando que maiores valores de ruído geram uma maior dispersão dos pontos de constelação, que causa o aumento do número de erros de transmissão, como esperado.

Noise Voltage=0 e Loop Bandwidth=0

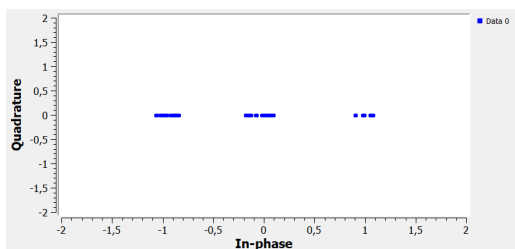


Figura 5: Constelação BPSQ

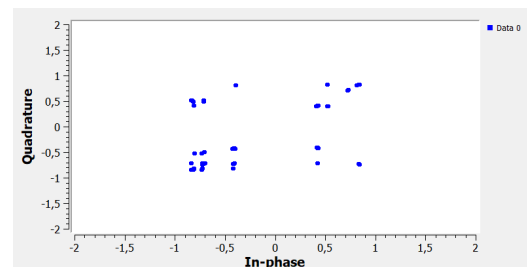
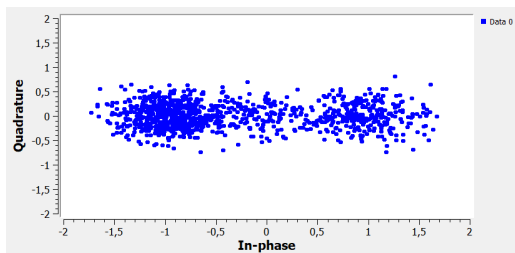
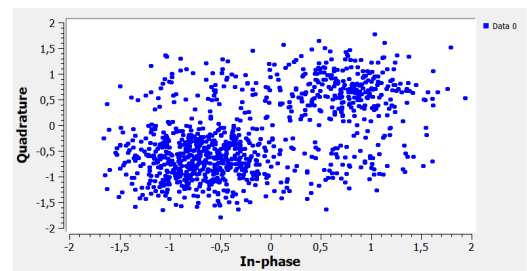
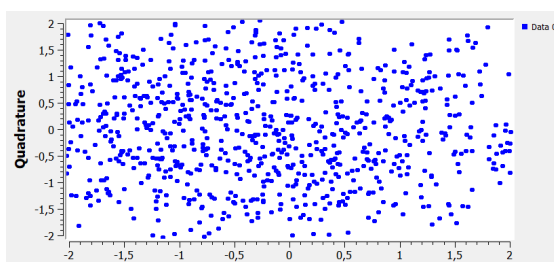
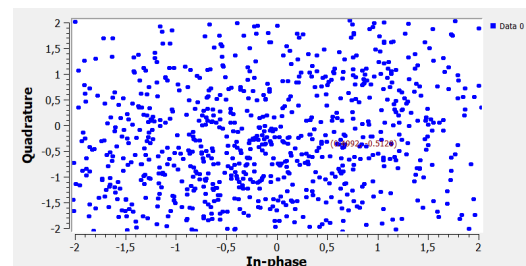


Figura 6: Constelação QPSK

Noise Voltage=1.5 e Loop Bandwidth=0.15

**Figura 7:** Constelação BPSK**Figura 8:** Constelação QPSK

Noise Voltage=4.5 e Loop Bandwidth=0.23

**Figura 9:** Constelação BPSK**Figura 10:** Constelação QPSK

Script e análise de erros

De forma a analisar o número de erros da desmodulação tanto na montagem do **BPSK** como na do **QPSK**, com o aumento das componentes *Noise Voltage* e *Loop Bandwidth* foi realizado o seguinte código. Este código faz parte de um script novo que está relacionado com o código .py criado pela

A nossa função `sim()` é responsável por simular um sistema de comunicação digital usando modulação BPSK/QPSK, com o objetivo de avaliar como o ruído e a largura de banda influenciam os erros na transmissão de dados. Para isso, definimos uma lista de valores de ruído e uma gama de larguras de banda a testar. Para cada combinação de ruído e largura de banda, a função chama a função `calc` que executa a simulação e calcula o número de bits diferentes, comparando o ficheiro de dados enviados com o ficheiro de dados recebidos. Estes erros são registados numa tabela que criámos para organizar os resultados. No final, a função guarda automaticamente os dados num ficheiros Excel, facilitando a análise posterior sobre o impacto das condições do canal de comunicação na precisão da transmissão.

```

9  OFFSET_ARQUIVO2 = 49 # colocar o atraso no arquivo 2
10 NUMERO_BITS = 216 # numero de bits de comparacao
11
12 def calc(arquivo1, arquivo2): # funcao que calcula o numero de bits diferentes
13     with open("BPSK_sent.dat", "rb") as arq1, open("BPSK_received.dat", "rb") as arq2:
14
15         bits_restantes = OFFSET_ARQUIVO2 - 8 * (OFFSET_ARQUIVO2 // 8)
16         arq2.seek(OFFSET_ARQUIVO2 // 8)
17
18         dados1 = arq1.read(math.ceil(NUMERO_BITS / 8) + 1)
19         dados2 = arq2.read(math.ceil((NUMERO_BITS - bits_restantes) / 8) + 1)
20
21         bits_diferentes = 0
22
23         for indice in range(NUMERO_BITS):
24
25             bit_dados1 = int(bool(dados1[indice // 8] & (1 << (7 - indice % 8))))
26             bit_dados2 = int(bool(dados2[(indice + bits_restantes) // 8] & (1 << (7 - (indice + bits_restantes) % 8))))
27             if bit_dados1 != bit_dados2:
28                 bits_diferentes += 1
29
30     return bits_diferentes
31

```

Figura 11: Função calc

```

def sim():
    app = QtWidgets.QApplication(sys.argv)
    N0 = [0, 0.5, 1.5, 2, 2.5, 3, 3.5, 4] #diferentes valores do noise voltage
    loop_BW = np.linspace(0,0.25,30) #diferentes valores do loop bandwidth
    ##f = open("data.csv", "w")
    #f.write("N,BW,bit_errors\n")

    data = {"BW/N": loop_BW}
    for N in N0:
        errors=[]
        for BW in loop_BW:
            tb = top_block_cls(BW, N) #inico da simulacao
            tb.start()
            tb.wait()
            print(f"Noise voltage: {N}, Loop bandwidth: {BW}")

            # ve o numero de erros
            erro = calc("bpsk_sent.dat", "bpsk_rec.dat")
            print(erro)
            errors.append(erro)
        data[str(N)]=errors
        #row += f",{erro}"
        #f.write(f"{N},{BW},{erro}\n")
        #f.write( f"\n")
    df = pd.DataFrame(data)
    df['Total Errors'] = df.iloc[:, 1:].sum(axis=1)
    df.to_excel("data.xlsx", index=False) #guarda como excel
    df.to_csv("data.csv", index=False) #tambem guarda como csv

    #f.close()
    signal.signal(signal.SIGINT, sig_handler)
    signal.signal(signal.SIGTERM, sig_handler)

if __name__ == "__main__":
    sim()

```

Figura 12: Função sim

Análise BPSK

BW/N	0	0.5	1	1.5	2	2.5	3	3.5	4	Total Errors
0	0	0	0	2	12	17	24	37	43	135
0,0086	0	0	0	2	12	17	25	35	44	135
0,0172	0	0	0	2	11	17	25	35	43	133
0,0259	0	0	0	2	11	17	25	35	43	133
0,0345	0	0	0	2	9	18	25	35	41	130
0,0431	0	0	0	2	9	17	26	35	40	129
0,0517	0	0	0	1	9	16	26	36	42	130
0,0603	0	0	0	1	9	15	27	39	43	134
0,0690	0	0	0	1	9	15	27	39	42	133
0,0776	0	0	0	1	9	15	26	40	42	133
0,0862	0	0	0	0	9	14	24	40	42	129
0,0948	0	0	0	0	7	15	23	39	42	126
0,1034	0	0	0	0	6	15	22	39	42	124
0,1121	0	0	0	0	6	15	22	38	42	123
0,1207	0	0	0	0	6	15	22	38	42	123
0,1293	0	0	0	0	6	15	22	38	41	122
0,1379	0	0	0	0	6	15	23	38	42	124
0,1466	0	0	0	0	6	15	22	38	42	123
0,1552	0	0	0	0	6	15	22	37	42	122
0,1638	0	0	0	0	5	15	22	35	42	119
0,1724	0	0	0	0	5	15	22	35	42	119
0,1810	0	0	0	0	5	15	23	35	42	120
0,1897	0	0	0	0	6	15	21	35	42	119
0,1983	0	0	0	0	6	15	22	36	42	121
0,2069	0	0	0	0	6	15	23	36	45	125
0,2155	0	0	0	0	5	14	23	36	49	127
0,2241	0	0	0	0	5	14	25	36	50	130
0,2328	0	0	0	0	6	14	25	36	74	155
0,2414	0	0	0	0	6	14	25	36	95	176
0,25	0	0	0	0	5	14	25	35	94	173

Tabela 1: Tabela de erros do BPSK

Análise QPSK

BW/N	0	0.5	1	1.5	2	2.5	3	3.5	4	Total Errors
0.0000	0	0	7	18	28	40	49	53	56	251
0.0086	0	0	7	18	27	41	49	54	56	252
0.0172	0	0	7	18	26	42	49	53	56	251
0.0259	0	0	7	17	26	42	49	53	55	249
0.0345	0	0	7	17	26	41	49	53	55	248
0.0431	0	0	7	16	25	42	49	54	55	248
0.0517	0	0	7	16	25	41	47	53	54	243
0.0603	0	0	7	16	24	40	46	51	54	238
0.0690	0	0	7	15	24	39	46	51	51	233
0.0776	0	0	6	15	25	39	47	50	52	234
0.0862	0	0	6	15	26	38	47	49	52	233
0.0948	0	0	4	15	26	37	44	51	54	231
0.1034	0	0	4	15	25	38	44	49	54	229
0.1121	0	0	4	15	24	39	45	49	53	229
0.1207	0	0	4	15	24	37	45	48	52	225
0.1293	0	0	2	17	24	37	43	49	50	222
0.1379	0	0	2	16	24	36	46	48	53	225
0.1466	0	0	2	14	25	36	46	51	57	231
0.1552	0	0	2	13	29	38	46	53	58	239
0.1638	0	0	2	13	29	39	46	54	59	242
0.1724	0	0	2	12	29	39	47	52	58	239
0.1810	0	0	2	11	30	39	48	55	59	244
0.1897	0	0	2	11	29	39	49	55	60	245
0.1983	0	0	2	12	30	39	49	56	62	250
0.2069	0	0	3	12	30	40	48	56	63	252
0.2155	0	0	4	13	30	40	48	56	63	254
0.2241	0	0	4	14	31	40	48	57	62	256
0.2328	0	0	4	14	31	39	49	56	61	254
0.2414	0	0	4	13	32	40	48	55	60	252
0.2500	0	0	4	13	33	40	50	54	62	256

Tabela 2: Tabela de erros do QPSK

Conclusão

Como mencionado anteriormente o desmodulador **BPSK** opera num espaço bidimensional, existem dois símbolos, um para o bit zero e outro para o bit 1, sendo o bit 1 representado por 1 e o bit zero representado por -1. Já a desmodulacao do **QPSK** opera num espaço quadridimensional, há 4 símbolos possíveis [00,01,11,10].

Gráficos: Os gráficos 11 e 12 representam o número de erros à medida que o *Noise voltage* vai aumentando, fizemos diferentes séries que correspondem aos diferentes valores de *Loop Bandwith*. Já os graficos 13 e 14 representam o número de erros com o aumento do *Loop Bandwith*, mais uma vez fizemos diferentes séries que correspondem aos diferentes valores de *Noise voltage*.

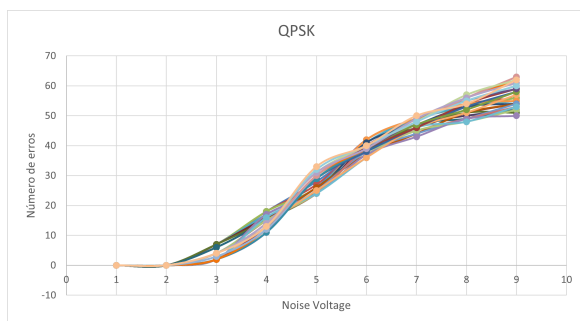


Figura 13: QPSK - Noise Voltage

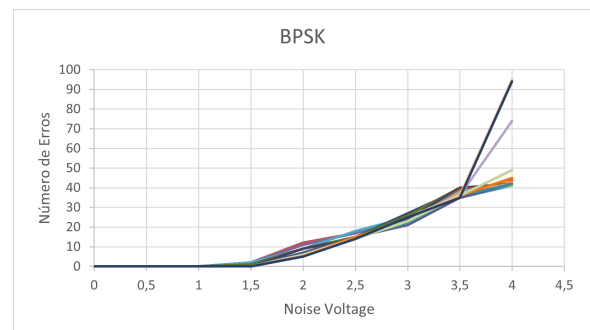


Figura 14: BPSK - Noise Voltage

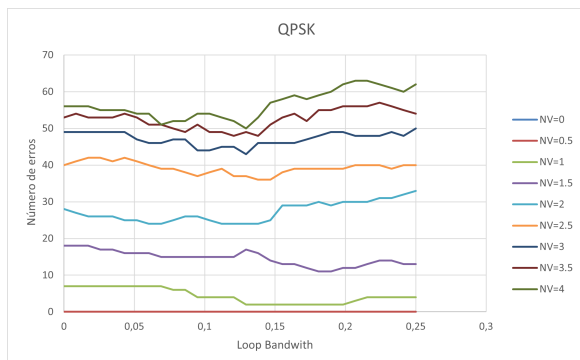


Figura 15: QPSK - Loop Bandwith

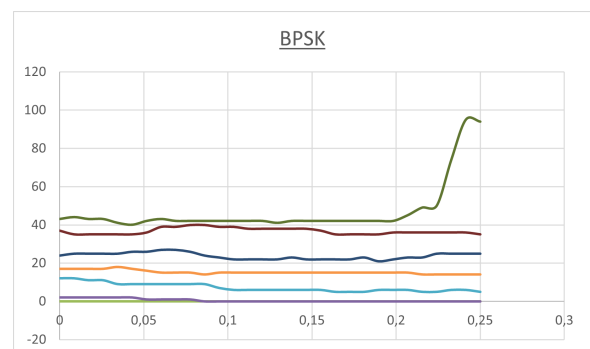


Figura 16: BPSK - Loop Bandwith

Resultados: Pelos gráficos 11 e 12 conseguimos afirmar que **BPSK** apresenta um aumento mais acentuado em relação aos valores de BW/N e atinge valores máximos mais elevados, enquanto **QPSK** tem um número de erros que se mantém relativamente estável em relação ao aumento da largura de banda. Ao mesmo tempo nos gráficos 13 e 14 observamos que para valores pequenos do *Noise voltage*, o número de erros tende a diminuir com o aumento do *Loop Bandwith*.

Assim, é de notar que em ambos os gráficos do desmodulador **BPSK**, figuras 12 e 13, para valores elevados de *Noise voltage* e de *Loop Bandwith* há um aumento significativo do número

de erros. Isto ocorre a partir de valores de *Loop Bandwidth* dos 0.23, quando o valor do *Noise voltage* é igual a 4. O mesmo não acontece quando estamos no desmodulador **QPSK**.

Capacidade de transmissão: Sendo que o **BPSK** transmite um único bit por símbolo, a taxa de transmissão de dados é diretamente proporcional à frequência do sinal portador, esta é assim limitada, pois só se pode transmitir 1 bit por ciclo. Já o **QPSK** pode transmitir 2 bits por símbolo, o que significa num único ciclo do sinal portador, podem ser enviados dois bits de informação. Assim a capacidade de transmissão no desmodulador **QPSK** é o dobro da capacidade de transmissão do **BPSK**.

Desta forma podemos calcular a eficiência espectral, sabendo que:

$$B = \frac{R_s}{2}(1 + \alpha) \quad (1) \quad R_b = \log_2 M \cdot R_s \quad (2)$$

E que a largura de banda B_t é

$$B_{RF} = 2B = R_s \cdot (1 + \alpha) \quad (3)$$

Então, a eficiência espectral é a seguinte:

$$\eta = \frac{R_b}{B_{RF}} = \frac{\log_2 M \cdot R_s}{R_s \cdot (1 + \alpha)} = \frac{\log_2 M}{(1 + \alpha)} \quad (4)$$

De acordo com o que foi referido anteriormente $M=2$ para o **BPSK** e $M=4$ para o **QPSK**.

Assim:

$$\eta_{BPSK} = \frac{\log_2 2}{(1 + \alpha)} = \frac{1}{1 + \alpha} \quad (5) \quad \eta_{QPSK} = \frac{\log_2 4}{(1 + \alpha)} = \frac{2}{1 + \alpha} \quad (6)$$

A eficiência espectral do desmodulador **QPSK** é superior à do **BPSK**, conseguimos transmitir dois bits por símbolo tendo esta montagem assim vantagens em relação à montagem **BPSK**.